



UNIVERSITY OF PISA

COMPUTER ENGINEERING MASTER DEGREE

Performance Evaluation of Computer Systems and Networks

Fragmenting wireless link

Professors:

Giovanni Stea

Giovanni Nardini

Group Members:

Mykhaylo Semenyshen

Tommaso Ottali

Luca Di Gasparro

ACADEMIC YEAR 2025/2026

Contents

1	Description	3
2	Objectives	3
3	Key Performance Indices	3
4	Model and Assumptions	3
5	Factors and Parameters	4
6	Simulation Tool and Implementation	4
6.1	Data Analysis Methodology	5
7	Verification	5
7.1	Internal Consistency Check	5
7.2	Degeneracy	6
7.3	Consistency	6
7.4	Continuity	7
7.4.1	Packet Size Sensitivity	7
7.4.2	Inter-Arrival Time Sensitivity	7
7.4.3	Byte Error Rate (α) Sensitivity	8
8	Validation	8
9	Experiment design	9
9.1	Warm-up and simulation length estimation	9
9.2	Design	9
9.3	Scenarios	10
9.3.1	Bluetooth Industrial	10
9.3.2	Bluetooth Audio Streaming	11
9.3.3	Bluetooth Sensors	12
10	Data Analysis	13
10.1	Statistical Significance of the Results	13
10.2	Instability of <i>Industrial</i> , $P=200$, $\alpha = 0.01$	14
10.3	System Utilization Analysis	14
10.4	End to End delay	15
10.5	Fairness and Predictability Analysis	18
10.5.1	CoV in the <i>Sensors</i> configuration	18
10.5.2	Cov in <i>Industrial</i> configuration	21
10.5.3	CoV and system stability	22
10.5.4	CoV analysis conclusions	23
10.5.5	Lorenz Curve Analysis	24
10.6	Bluetooth AudioStreaming	25
10.7	Queue Length estimation	26

A	Verification results	29
B	Theoretical Queuing Model Computations	31
B.1	Input Parameters	31
B.2	Fragment Statistics	31
B.2.1	Case 1: Fixed Packet Size ($S_{\max} = S_{\min}$)	31
B.2.2	Case 2: Uniform Distribution ($S_{\min} < S_{\max}$)	31
B.3	Service Time (t_s)	32
B.3.1	Transmission Cycle	32
B.3.2	Error and Retransmission Statistics	32
B.3.3	Service Time Moments	33
B.4	System Performance Metrics	33
B.4.1	Mean Waiting Time ($E[W]$)	33
C	Validation results for simulated scenarios	34

1 Description

This project aims to perform a comprehensive analysis of a wireless transmitter-receiver system that utilizes fragmentation of application-level packets.

2 Objectives

The objective of this study is to identify the optimal size of the transmitted fragments to minimize end-to-end packet delay and system utilization; to evaluate system fairness, and to estimate the queue length of a real transmitter buffer for each simulated scenario, to assess its feasibility.

3 Key Performance Indices

Therefore, the main areas of interest are:

- End-to-end packet delay
- Fragments in queue
- System utilization
- Fragment retransmissions

4 Model and Assumptions

The system is modeled as an M/G/1 system with group arrivals, caused by the fragmentation. The following assumptions have been made:

- Infinite queue, so that no packet or fragment can be discarded
- The delay due to fragmentation at the transmitter is negligible
- The delay due to reassembly of fragments at the receiver is negligible

The first assumption is admittedly unrealistic, as practical systems have finite buffers. However, we adopt it to evaluate system performance without buffer-induced effects. At the end of this work, we use the measured queue lengths across scenarios to dimension appropriate buffers and assess whether realistic buffer sizes would suffice.

The second assumption is justified by considering that when a packet arrives to an empty queue, it must begin transmission immediately. Since the packet resides in memory accessible via a data bus, extracting the first fragment takes negligible time compared to the transmission cycle (fragment transmission plus ACK/NACK reception). During this cycle, the remaining fragments can be prepared. The critical delay is thus the first fragment extraction: if this is much

smaller than the transmission cycle time, it becomes negligible. This assumption holds for modern systems but may fail for extremely slow memory buses combined with very high-speed links—characteristics typical of legacy systems. When the queue is already occupied, this concern disappears entirely, as fragmentation can proceed in parallel while other fragments are being transmitted. The third assumption follows similar reasoning: reassembly operations (fragment buffering and packet reconstruction) at the receiver involve memory operations that are orders of magnitude faster than network transmission times. This holds for systems with adequate processing capabilities and memory bandwidth.

5 Factors and Parameters

The factors involved include:

- Byte Error Rate α
- Fragment Size P

The parameters include:

- Header Size H
- Wireless Link Bandwidth C
- Acknowledgment Size A
- Interarrival Time of Packets $E[T]$
- Packet Size Range S_{min} and S_{max}

The parameters remain constant within each configuration, but they vary across different configurations.

The fragment error probability is given by

$$p = 1 - (1 - \alpha)^{P+H}$$

6 Simulation Tool and Implementation

The simulation tool utilized for this project is **OMNeT++**, an open-source, modular framework designed for discrete-event system modeling and simulation. A simple network was implemented, consisting of two modules: the *Tx* and *Rx* modules. The OMNeT++ *Packet* class was employed for exchanging messages between the sender and the receiver due to the ease of embedding values within *Packet* objects. These values are exchanged between the modules for statistical computations, such as the *queueArrivalTime* of fragments, which is used to compute the end-to-end delay of packets, or the *last* information, which indicates to the receiver when the final fragment of a packet has been received.

To send fragments and acknowledgments, the *sendDirect()* function was used. This function effectively models the delay caused by channel propagation time (by taking it as an argument), which is deterministic in this case. It also eliminates the need for timers before transmission, which would be initialized to the delay of the transmitted data, thereby simplifying the code structure.

6.1 Data Analysis Methodology

We verified that the variance for each scenario was finite: this was done by plotting the sample variance as a function of the sample size. An example plot can be found in Figure 1. Each scenario was simulated with a count of 30 replicas, each of which was initialized with different RNG seeds to ensure statistical independence. Therefore, confidence intervals for the population means could be reliably computed using the Standard Normal distribution.

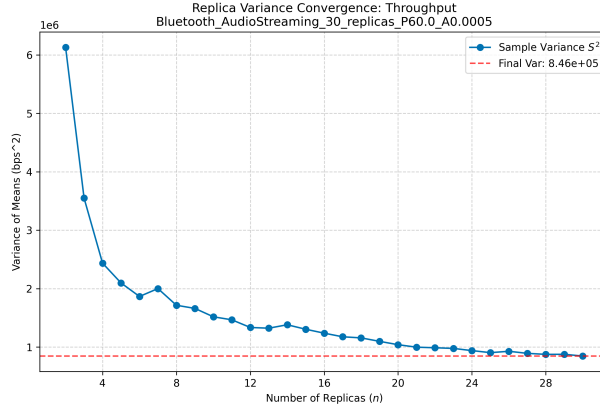


Figure 1: Finite variance convergence check for the Audio Streaming scenario ($P = 60$, $\alpha = 0.0005$). The stabilization of the sample variance S^2 as $n \rightarrow 30$ confirms the statistical validity required for confidence interval estimation.

7 Verification

This section describes the verification strategy employed to ensure the correctness of the implemented simulator.

7.1 Internal Consistency Check

A sanity test validates that the implementation correctly satisfies fundamental queuing theory laws, providing confidence in the simulation’s correctness. Violations of these invariants indicate bugs in either the statistical measurement infrastructure or the core queuing logic, preventing analysis of invalid data.

Only general laws involving mean values and stability conditions were tested, as the model has not yet been validated. The tested laws are:

- **Little’s Law:** $E[N] = \lambda E[R]$ and $E[N_q] = \lambda E[W]$.
- **Utilization Law:** $\rho = \lambda E[t_s]$
- **Additive properties of mean values:** $E[R] = E[W] + E[t_s]$ and $E[N] = E[N_q] + \rho$.
- **Flow consistency:** This refers to the alignment between application throughput (defined as the number of application bits correctly acknowledged by the receiver, excluding header bits) and the bit arrival rate at the transmitter, from the application level, defined as $\lambda = \frac{\bar{S} \cdot 8}{E[T]}$, where $\bar{S}$ is the mean packet size in bytes, and $E[T]$ is the mean interarrival time.

Errors were near 0 across all scenarios tested. An example of output can be found in Table 3 in Appendix A.

7.2 Degeneracy

The simulator was tested under limit conditions to ensure physical consistency:

- **Zero Error Rate ($\alpha = 0$):** As expected, no retransmissions occurred, and the service time was strictly equal to the transmission cycle time.
- **Total Error Rate ($\alpha = 1$):** The system resulted in zero throughput as no fragments could be successfully acknowledged.
- **High Corruption Probability:** By fixing packets and fragment sizes at 10 kB, the theoretical fragment corruption probability reached 99.996%; we found that no fragments were acknowledged.

These three tests confirmed that the simulator handles extreme boundary conditions correctly.

7.3 Consistency

We evaluated the simulator’s response when the theoretical input bit rate remains constant across different configurations. In the first scenario, packets fell within a range of [500-800]B with an inter-arrival time of 1 second. The second scenario doubled both the packet size range and the inter-arrival time. A deviation of approximately 0.07% in throughput was observed, thereby confirming our expectations.

7.4 Continuity

We conducted a continuity analysis by applying marginal increments to three primary input parameters: packet size, inter-arrival time, and byte error rate. This test verifies that the system evolves monotonically, ensuring that small input perturbations yield predictably small output variations. In the first two tests, the system was significantly under-utilized (around 0.5%), allowing us to minimize retransmission effects on parameters and focus on the testing objectives.

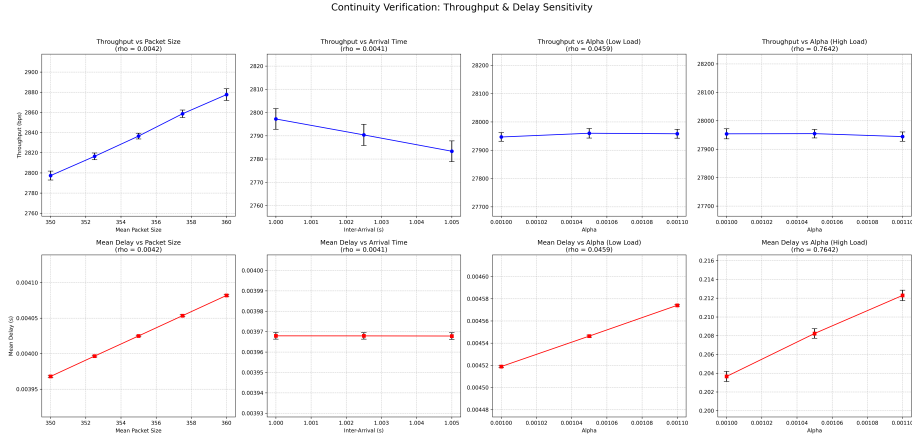


Figure 2: Continuity analysis results showing the sensitivity of throughput and delay to variations in Packet Size, Inter-Arrival Time, and Byte Error Rate (α). Results plotted with 95% confidence intervals.

7.4.1 Packet Size Sensitivity

For the packet size, we increased the mean by 0.7% for each step, reaching a maximum deviation of 2.8% from the starting configuration (see Table 4 in Appendix A). Throughput exhibited a linear increase, aligning with the byte arrival rate, while the end-to-end delay also rose linearly due to the increased volume of bytes being transmitted.

7.4.2 Inter-Arrival Time Sensitivity

For the inter-arrival time, we applied a 0.25% increase per step, with a maximum deviation of 0.5% (see Table 5 in Appendix A). Throughput decreased linearly for the same reasons mentioned previously, whereas end-to-end delay remained constant, as it is unaffected by arrival rates (in this specific case of underutilization).

7.4.3 Byte Error Rate (α) Sensitivity

Finally, we evaluated sensitivity to the error probability parameter under two distinct loading conditions to assess the system’s stability boundaries (see Tables 6 and 7 in Appendix A). We increased the error probability by 5% per step, with a maximum deviation of 10%. In both cases, assuming the system is stable, throughput remained constant for the reasons cited earlier. At low load (5% utilization), delay increased by approximately 1% due to more corrupted fragments, but did not spike, as underutilization minimized queuing delay. Conversely, under high load (75% utilization), delay increased by up to 5% due to queuing effects, reflected in the mean number of fragments in the queue rising from nearly 0 to over 6.

8 Validation

The system is considered as an M/G/1 with group arrivals, whose statistics computations can be found in Appendix B. Comparing results obtained via these theoretical computations and those measured in the simulation, the discrepancies were minimal ($< 5\%$), as shown in Table C.1. However, in specific scenarios, such as the *Bluetooth Sensors* configuration, notable errors were observed, ranging from 15% to 70% or more, as shown in Table C.2.

The physical justification for this divergence lies in the fact that theoretical computations are based on an average, “statistical” fragment and do not account for the heterogeneity between this mean fragment and a full-sized fragment. These errors arise because, in this configuration, the number of fragments per packet is quite low, as the size of the packets is comparable with P . Consequently, this results in a couple of full-sized fragments and a nearly empty tail fragment (In Table C.2, $E[K] = 1.5$: $\overline{S_{\text{frag}}} = 40$, which is $2/3$ of $P = 60$).

This leads to a higher corruption probability that the simplified theoretical model fails to capture. Consequently, the waiting time—from which other queuing metrics are derived—is severely underestimated, causing the error to propagate. In these specific cases, the p_{diff} metric reaches values exceeding 20-30%. These large, full fragments effectively trigger a Head-of-Line blocking problem, obstructing all subsequent fragments for a significantly longer duration compared to mean-sized fragments. Physically, in the simulation, a single full fragment undergoing multiple retransmissions heavily penalizes all subsequent fragments in the queue, including small tail fragments that would otherwise require only a single, rapid transmission, heavily increasing their waiting time.

As shown in the waiting time formula (Equation 14), indeed, the source of error is the mean residual service time (computed in Equation 13). This term depends heavily on the variability of the service time (i.e., its second moment, related to its variance), which is not accounted for correctly by a mean theoretical fragment.

On the contrary, the mean service rate shows low error and the mean retransmissions per fragment actually match perfectly: for these linear mean-values,

full fragments are actually balanced by the tail ones, and a higher service time requested by a full fragment, or a higher number of retransmissions required by it do not influence subsequent smaller fragments’ service times or retransmissions, as they are independent. As a consequence, the theoretical mean fragment is a correct approximation.

9 Experiment design

9.1 Warm-up and simulation length estimation

The warm-up duration was estimated using a mean-over-time analysis on the complete set of 30 replicas. In Figure 3 only four of them are plotted for simplicity. For each replica, we computed the cumulative mean of the response time using all samples contained in expanding time windows starting from zero. The window width was increased in 1-second steps, with the x-axis representing window width and the y-axis representing the mean response time computed over that window.

As shown in Figure 3, the mean-over-time curves initially exhibit large fluctuations and significant differences among replicas, indicative of the transient phase. After a certain time, all curves stabilize around a common value with only small fluctuations, indicating that steady-state conditions have been reached. Since this study evaluates steady-state performance, we discarded the initial portion of the simulation corresponding to the transient phase.

For each scenario, the warm-up time T_w was selected as the earliest time after which the mean-over-time curves of all replicas remain stable and converge. All performance metrics were then computed using only samples collected after T_w .

After determining T_w , the total simulation length was chosen such that the remaining steady-state portion ($T_{sim} - T_w$) contains sufficient observations to ensure stable estimates of performance metrics. In practice, the simulation length was increased until the estimated means and confidence intervals showed negligible changes with further extension. This ensures that reported results are based on steady-state data with adequate statistical accuracy.

9.2 Design

The scenarios have been designed to ensure that the simulation configurations are both realistic and representative of real-world use cases. The primary objective was to develop a diverse set of configurations—varying in traffic load, environmental noise, and packet arrival frequency—to stress the system under a wide range of operational conditions. This way, different applications prioritize different performance metrics. By exploring the different tradeoffs required for each of these, we can obtain a deeper understanding of how the system responds to a different set of factors.

Since there are only two factors (i.e., α and P), a full factorial analysis will be

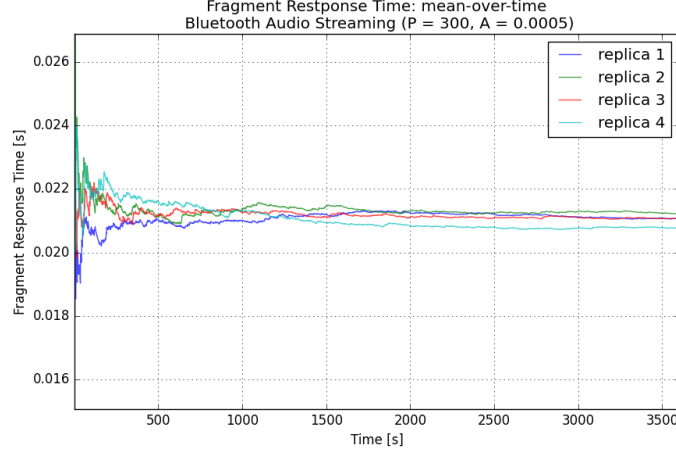


Figure 3: Mean-over-time of the response time for four independent replicas, computed using expanding time windows.

exploited. Moreover, each use case is designed for a typical environment, hence the values that α can assume are limited to a couple.

9.3 Scenarios

The Bluetooth protocol suite was selected as the primary reference for this study due to its versatility across diverse use cases, each imposing distinct operational requirements. The Bluetooth specifications offer two primary protocol implementations [8].

The Low Energy Protocol is designed for applications requiring minimal power consumption, reduced complexity, and lower implementation cost, compared to traditional BR/EDR protocols. It supports transmission rates of 1Mb/s and it enforces a 255-byte payload constraint and requires an 11-byte header [10]; legacy versions enforce a 20-byte payload constraint [6]. This protocol is well suited for IoT devices and microcontrollers, which often stick to legacy versions due to long product life cycles and compatibility [24].

The Bluetooth Basic Rate and Enhanced Data Rate protocols are designed for systems requiring higher data transmission rates without very restrictive power consumption constraints. The Basic Rate protocol supports a rate of 1Mb/s, while the Enhanced Data Rate reaches 2Mb/s and 3Mb/s. They enforce a 348-byte payload constraint and require a 16-byte header [9]. A typical use case is high-quality audio streaming with wireless headphones.

9.3.1 Bluetooth Industrial

This configuration simulates an industrial IoT node performing predictive maintenance on critical rotating machinery (e.g., turbines, pumps, compressors).

Table 1: Simulation Configuration Parameters for Bluetooth Scenarios

Parameter	Sensors	Industrial	Audio Streaming
<i>Base Configuration</i>	Header: 11B, Bandwidth: 1Mbps, Ack: 10B		
Payload Size (B)	20 – 100	6000 – 8000	500 – 2000
Inter-arrival Time (s)	0.002 (500 Hz)	0.5 (2 Hz)	0.02 (≈ 43 Hz)
Error Rate (α)	0.005, 0.01	0.005, 0.01	0.0005, 0.001
Fragment Size (B)	20, 40, 60, 80	20, 60, 120, 200	60, 120, 240, 300

The node captures raw vibration waveforms for remote Fast Fourier Transform (FFT) analysis to detect bearing faults, imbalance, misalignment, and other mechanical anomalies [23]. To analyze vibration frequencies up to 3-4kHz, the Nyquist theorem requires a minimum sampling frequency of 6-8kHz. Following standard industrial practice in condition monitoring systems, an FFT length of 4096 samples is commonly used, balancing frequency resolution with computational efficiency [5]. Using 4096 samples at a 16-bit resolution yields 8192 bytes of raw data. Therefore, with some approximations, accumulating 4096 samples at a sampling rate of 8192Hz requires about 0.5 seconds. The 6000-8000 range accommodates lower sampling rates and buffer sizes.

These nodes operate in a noisy environment due to electromagnetic interference from high-voltage motor drives and severe multipath fading caused by the dense metallic infrastructure, typical of a factory environment.

This scenario is designed to understand how the system evolves with high noise and the impact of fragmentation on this effect, considering that the large size of packets implies a large number of fragments generated.

9.3.2 Bluetooth Audio Streaming

AAC is a variable bitrate block-based codec where each block decodes to 1024 time-domain samples. For standard music playback at CD quality, the sampling rate is 44.1 kHz [15] [12]. At a 44.1 kHz sampling rate, accumulating 1024 samples requires approximately 23 ms ($1024 \div 44,100 \text{ Hz} \approx 0.023 \text{ s}$). Traditional AAC protocol packets range from 368 bytes (with an optimized rate of 128 kbps for silence or simple sounds [2] [4]) to 1000 bytes (with a full rate of 320 kbps [13]). However, the lowest bound is rarely utilized in practice, as slightly higher rates are typical (around 256 kbps for acceptable music quality [3]). Additionally, mixed-codec scenarios may employ higher-quality codecs with protocol overhead from A2DP framing, L2CAP headers, and AVDTP transport layers. The 500-2000B range therefore models realistic variable bitrate encoding across multiple quality levels and codec implementations, capturing both standard AAC operation and peak conditions with protocol overhead [7].

The byte error probability is kept very low as this is a short-range communication, typically in a consumer environment with moderate radio frequency interference (e.g., Wi-Fi).

This scenario is designed to act as a benchmark for protocol efficiency in a high-throughput, low-noise regime. Unlike the *Industrial* (where performance is limited by retransmissions) or the *Sensors* (where performance is limited by header overhead relative to payload) scenarios, Audio Streaming represents a "middle ground" where packet sizes are moderate. This allows us to isolate the impact of the fragment header overhead H on the system utilization and delay, effectively treating it as a baseline for ideal channel conditions.

9.3.3 Bluetooth Sensors

This configuration simulates a network of high-performance wearable sensors [20] [22] operating on the Nordic nRF52840 SoC, which supports the Bluetooth protocol [19]. The system utilizes a 0.002s (500 Hz) packet inter-arrival time to capture rapid kinematic events [16]. The 20-byte payload represents a single raw IMU (Inertial Measurement Unit) sample, as defined in [18], which specifies 18 bytes of raw data (Accelerometer, Gyroscope, and Compass at 16-bit resolution) plus other headers. The upper limit of 100 bytes models a "batched" transmission where approximately 5 such samples or a group of different values, are aggregated into a single packet for transmission [17].

The byte error rate is medium-to-high because of the body-shadowing effect [11]. A higher error rate ($\alpha = 0.01$) is considered to take into account group activities, in which other wireless devices and sensors may interfere with one another. This scenario is designed to study how the system behaves under a low-fragmentation regime: each packet is fragmented in a very low number of fragments; therefore, it could be interesting to see how fragment statistics propagate to packet ones in comparison with the preceding configuration.

10 Data Analysis

As the first step in the analysis of the simulation data, it is interesting to see how the corruption probability of fragment varies with both P and α . As we can see in Figure 4, there is a great difference between low α (0.0005, 0.001) and high α (0.005, 0.01): in the first case, the function is linear with P , in the second it is non-linear and assumes values much higher. As a first observation, in low α scenarios, increasing P is a safer option than in high α ones, as it only slightly goes past 20% for the tested P .

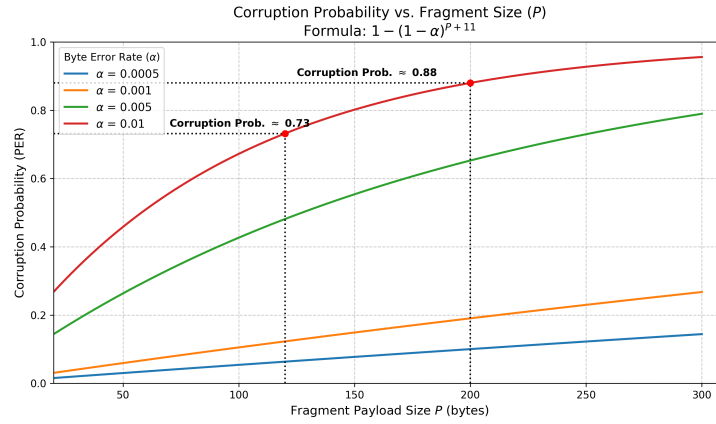


Figure 4: Fragment corruption probability as a function of α and P . The dotted guidelines highlight the critical increase in corruption probability for the *Industrial* scenario ($\alpha = 0.01$) when moving from a stable configuration ($P = 120$) to an unstable one ($P = 200$).

10.1 Statistical Significance of the Results

All metrics reported throughout this analysis include 95% confidence intervals computed across the 30 replicas using the methodology described in Section 6.1. For the mean delay, these represent standard statistical confidence intervals. For metrics such as standard deviation and Coefficient of Variation, we apply the same computational approach for consistency; however, we acknowledge these should be interpreted as measures of coherence across replicas rather than confidence intervals in the strict statistical sense, given the analytical complexity of computing true confidence intervals for such derived statistics [14] [21]. These intervals are consistently orders of magnitude smaller than the measured values (typically 2-4), impacting for less than 3% of the mean value), confirming high statistical significance. For brevity, individual confidence intervals are not reported.

10.2 Instability of *Industrial*, P=200, $\alpha = 0.01$

As observed, exceeding the saturation point in high-stress scenarios leads to severe performance degradation. This is evident in the *Industrial* scenario with $\alpha = 0.01$ and P=200, where the elevated packet error rate triggers an excessive number of retransmissions. Consequently, the effective service time for a fragment increases drastically, causing the queue depth to grow unboundedly. The system utilization approaches unity, indicating that the system fails to reach a steady state as defined by queuing theory; under these conditions, standard performance analysis becomes meaningless as the system is essentially unstable. This instability is further evidenced by the fragment response time PDF, which is highly skewed to the right and assumes values in order of hundreds of seconds, as seen in Figure 5. The contrast in stability between P=120 (stable) and P=200 (unstable) highlights the non-linear sensitivity of the fragment corruption probability to fragment size: above a critical threshold, the probability of corruption rises so steeply that the effective transmission rate drops below the application's arrival rate, leading to system instability (as shown in Figure 5).

As a note, in the following sections, when discussing a specific configuration, the expressions *high noise* and *low noise* refer to the maximum and the minimum value of α for the particular configuration object of discussion, respectively.

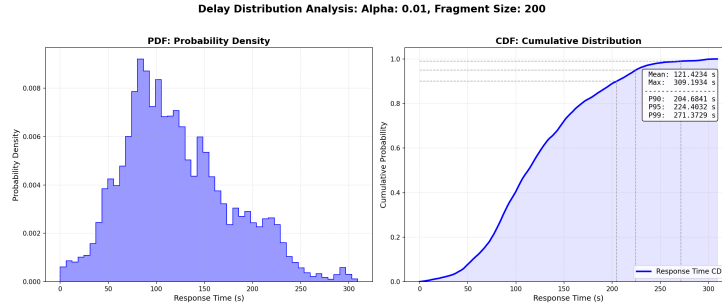


Figure 5: PDF of the unstable *Industrial* scenario, with P=200 and $\alpha = 0.01$.

10.3 System Utilization Analysis

The system utilization exhibits an expected U-shape with minimum utilization for an optimal fragment size, which suggests that system has reached a saturation point, over which no more performance gains are observed:

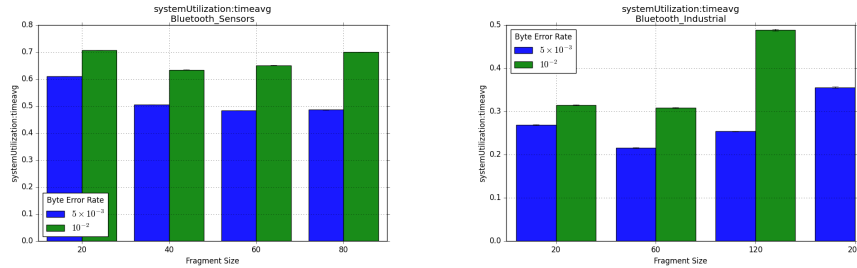
- Pre saturation behavior: the transmitter spends significant time transmitting headers (e.g., with P=20, worst case in *Industrial* and *Sensors* and the standard H=11, approximately half of the bytes transmitted are headers, therefore half of the transmission time), and waiting for receiver

ACKs due to high fragments count per packet. Therefore, increasing the fragment size reduces these overheads.

- Post saturation behavior: the utilization rises due to channel congestion from retransmissions, negating further protocol efficiency gains. This rise rate increases with noise level, consistent with trends observed in end-to-end delay and predictability metrics.

Therefore, the optimal point (saturation point) is the one which minimizes the protocol overhead without excessive retransmission penalties. Of course, the U-shaped is shifted to the left (lower P values) with a higher noise, and to the right (higher P values) with a lower one (as shown in Figure 6). The exact same behavior is reflected across all configurations: this was expected, since it relates to general concepts of efficiency and system equilibrium.

From this analysis, we understand that even in moderately harsh environments as in *Industrial*, it makes sense to use fragments larger than 20-bytes, which was the BLE legacy maximum payload constrain, as introduced before. Interpreting the system utilization as a measure of an IoT device energy efficiency (i.e., the amount of time it spends serving packets and therefore consuming its battery), we may conclude that a lower utilization implies better power saving: therefore, this reason adds to the preceding one, and even legacy devices may benefit from this newer version of the protocol.



(a) Utilization in *Sensors* configuration. (b) Utilization in *Industrial* configuration. Saturation at P=60 for low noise, and at P=40 for high noise. Utilization in *Industrial* configuration. Saturation at P=60 for both the configurations.

Figure 6: These figures show the utilization U-shape and the higher spikes with larger fragments and higher noise.

10.4 End to End delay

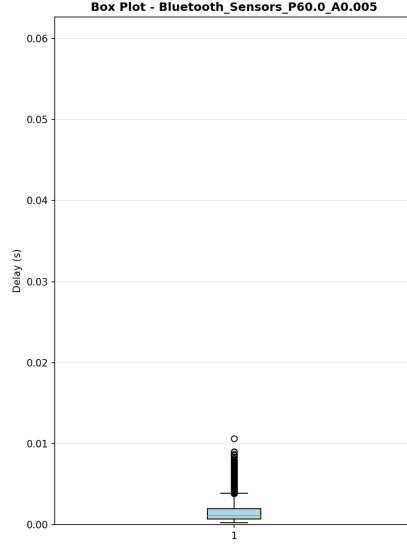
End-to-end delay is determined by two factors: the amount of time wasted due to protocol overheads as described above, and the time spent retransmitting the same fragments due to corruption. Therefore, it is expected that it follows the same U-shape described for the system utilization.

Taking the *Sensors* low noise configuration as a reference, the following was seen when switching between different P values:

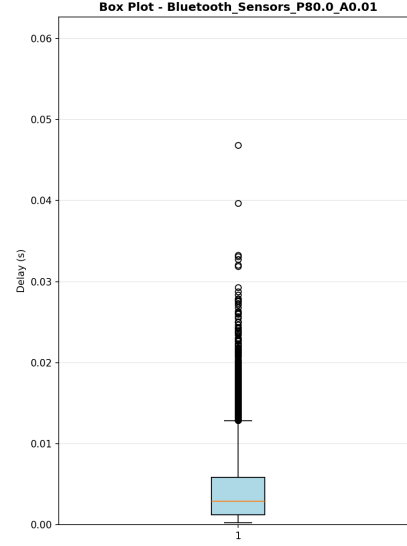
- $P = 20 \rightarrow 40$: mean delay reduces from 2.26ms to 1.58ms (30% reduction) and the 99th percentile shows equivalent 30% reduction. As a consequence, the whole distribution shifts towards lower values.
- $P = 40 \rightarrow 60$: mean delay decreases only by 5%, indicating approach to saturation point (lower bound for packet delay). However, the 99th percentile increases by 0.95%, the standard deviation grows by 2.15%. This means that the retransmission begins outweighing protocol efficiency gains, adding variability to service time and creating heavier distribution tails.
- $P = 60 \rightarrow 80$: mean delay increases by 5%, the standard deviation increases by 16%, and the 99th percentile increases by 15%. There is a spike in the delay variability due to retransmissions now dominating. The whole distribution becomes increasingly right-skewed with increasingly heavier tails.

So, beyond the saturation point, the system performance is increasingly defined by extreme outliers requiring multiple retransmissions, influencing all subsequently enqueued fragments and enhancing these negative effects. This trend is reflected across all configurations, and it is accelerated in the high noise scenarios, because the saturation point comes with lower P values and the higher corruption probabilities introduce higher variability in the system. In the *Sensors* high noise, indeed, it is located in $P=40$.

These trends can be graphically appreciated in Figure 8.

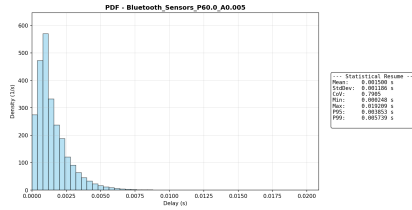


(a) Box plot for the optimal low noise scenario

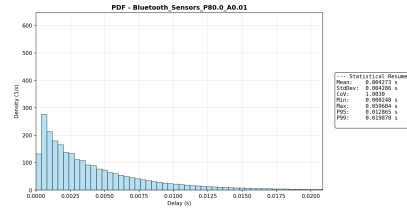


(b) Box plot for the worst high noise scenario

Figure 7: These box plots show the increasing number of outliers, whose values are increasingly further from the mean, as the error and P increase beyond saturation.



(a) PDF of end-to-end packet delay in the optimal low noise scenario.



(b) PDF of end-to-end packet delay in the worst high noise scenario.

Figure 8: These PDFs show how the distribution becomes increasingly right-skewed with heavier tails, as we move past the saturation point.

10.5 Fairness and Predictability Analysis

The analysis reveals a fundamental design trade-off: optimizing protocol efficiency through increased fragment size (P) inevitably compromises system delay predictability, and this correlation intensifies significantly in noisy environments.

10.5.1 CoV in the *Sensors* configuration

The Coefficient of Variation (CoV) provides a normalized measure of packet delay unpredictability. Using the *Sensors* configuration as reference:

- Low noise scenario: degradation is gradual and controlled, as CoV increases from 0.73 at P=20 to 0.87 to P=80.
- High noise scenario: degradation accelerates dramatically. CoV starts at 0.82 (comparable to worst-case low noise) and reaches 1.0 (P=80).

For a CoV of 1.0, the standard deviation equals the expected value, meaning natural system fluctuations occur on the same scale as mean performance. This matches the characteristics of an exponential distribution, indicating a memory-less-like state where past delay provides no information about remaining transmission time.

Following the Box Plot analysis in Figure 7, it reveals that higher CoV values correlate with an increased number of outliers positioned increasingly far from the mean as P increases. This trend is more pronounced in high noise conditions. As a consequence, mean delay alone inadequately characterizes system performance, as it fails to capture extreme jitter, a critical factor for real-time applications such as sensor networks, and the 99th percentile may be a better metric: that's the reason why comparison included that value before. From the box plots of Figure 7, it was already understandable. In Figure 9, it becomes evident. Therefore, the CoV exhibits a consistent increasing trend as P grows, but the underlying mechanisms causing this differ below and above the saturation point. The key is understanding how fragment-level statistics propagate to packet-level statistics, as the packet delay is the sum of the service times of its composing fragments plus the waiting time of its first fragment.

CoV under the *Sensors* saturation point Below the saturation point, increasing P reduces the number of fragments per packet, improving efficiency while keeping retransmissions under control. From queuing theory, we define the response time as the sum of service time and waiting time:

$$E[R] = E[t_s] + E[W]$$

- $E[t_s]$: can be approximated with a deterministic value, as the number of retransmissions per fragment is very low and near 0 (so, just 1 transmission is mostly enough to receive an ACK).

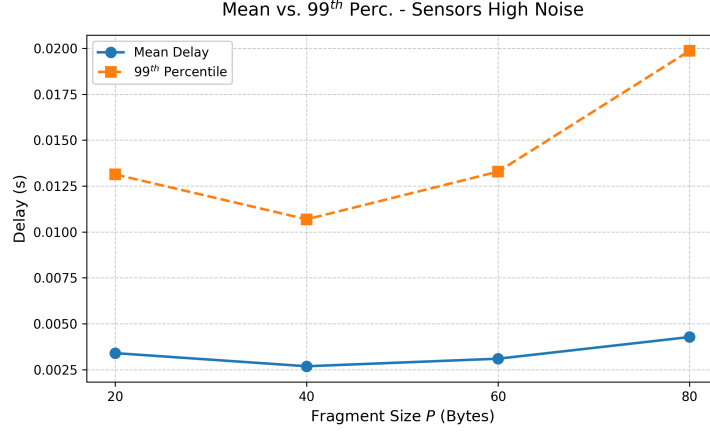


Figure 9: The different trend between the mean and the 99th percentile in high noise *Sensors*: then mean is no more an accurate representation of the system actual delay.

- $E[W]$: the main source of randomness is the waiting time, which is related to the group arrival phenomenon. By increasing P , we are reducing the number of fragments per packet, via bigger fragments: the service time increases slightly but still no considerable randomness is introduced, thus the approximation holds. The waiting time consequently decreases by having less fragments (there is near 40% reduction between $P=20$ and $P=40$ in *Sensors* low noise).

Consequently, it makes sense that the randomness in the response time decreases as well: indeed, for *Sensors* low noise, we have a reduction of 36% in the response time std. For the mean, by increasing the P size, the waiting time drastically reduces, while the service time increases (bigger fragments require more transmission time), so intuitively the mean cannot decrease as fast as the std: indeed, it decreases by 23%. Therefore, below saturation the CoV of the fragment response time decreases: the deterministic component grows, while the stochastic one shrinks: indeed, for $P=20$, 82% of the response time is due to waiting time; for $P=40$, it becomes 66%. We are reducing the baseline delay due to queuing caused by system inefficiencies.

By looking at the packet delay, however, the CoV is increasing: this means that the mean is decreasing faster than the std, the opposite of what we have found for fragments. For the high noise scenario, in which this effect is more accentuated, we have a 18% decrease in std and a 21% decrease in the mean. Physically, the mean packet delay scales linearly with the number of fragments (summing their service times up), hence it benefits both from a lower fragment response time and a lower number of fragments. On the other side, with some approximations, the std scales with the square root of the number of fragments, hence

it decreases slower. From an intuitive point of view it makes sense, as we have less statistical smoothing coming from variation compensation of the underlying response times due to a reduction in the number of fragments. Therefore, increased efficiency inherently causes a higher CoV. This effect is due to a very low number of fragments per packet in the *Sensors* configuration, between 1.5 and 3.

In other words, below saturation, the system is bottlenecked by queuing, not retransmission. When the number of fragments is reduced, the statistical smoothing effect occurring over the highly variable queue waiting times is reduced as well. Even though each fragment improves individually, having fewer of them means the packet-level predictability (CoV) worsens. With less statistical averaging across fragments, the persistent queue variance—which doesn’t scale down as efficiently as the mean—becomes proportionally more significant in determining total delay variability.

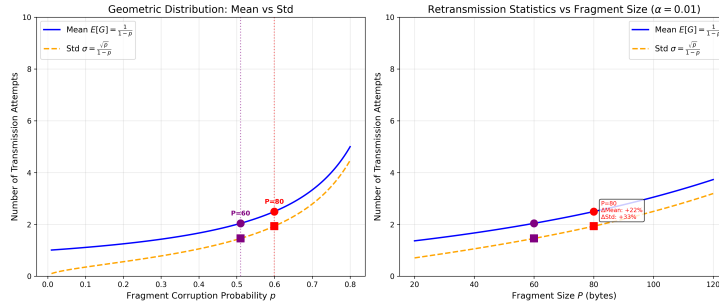


Figure 10: The std of a geometric grows faster than its mean by increasing P .

CoV beyond the *Sensors* saturation point Beyond saturation, the system becomes dominated by retransmission randomness. Retransmissions are distributed according to a geometric distribution, as explained in Appendix B. The geometric distribution’s variance is $\frac{p}{(1-p)^2}$, causing std to explode faster than mean, which is $\frac{1}{1-p}$. This is clearly seen in Figure 10. For example, from $P = 60 \rightarrow 80$ in fragment response time of high noise *Sensors*, the std increases by 52%, while the mean by 40%. Physically, an increase in retransmissions increases the service time: as a consequence, the waiting time of all subsequent fragments must increase as well. Variance grows faster than mean because variability now arises from both queue position (due to group arrivals) and service time variance (which was negligible below saturation but now dominates). The compound effect of these two sources causes std to grow substantially faster than mean. This is coherent with theoretical expectations, as the waiting time is influenced by the residual service time.

The same trend is reflected in packet delay: from $P = 60 \rightarrow 80$, the mean increases by 38%, while the std by 51%. One may expect the std to grow more due to the above considerations about statistical smoothing, but for these relatively

high values of P , the mean number of fragments is near 1, so the statistics must resemble very close those of a single fragment. This trends are clearly illustrated in Figure 11.

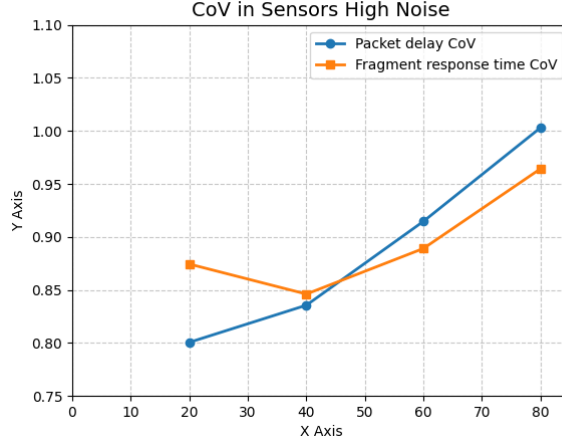


Figure 11: Comparison of CoV for Fragment Response Time vs. Packet Delay in the *Sensors* High Noise scenario. Note how the Packet CoV grows faster than the Fragment CoV beyond the saturation point ($P = 40$), confirming the loss of statistical smoothing.

10.5.2 Cov in *Industrial* configuration

Now, another analysis should be done on the other scenarios, for which the high number of fragments per packet may change some dynamics. No significant differences were found between *Industrial* and *Audio Streaming*, speaking of general trends, hence the analysis will focus on the *Industrial* scenario.

Packet delay *Industrial* CoV analysis A first consideration regards the packet delay CoV, which assumes values in the range $[0.3, 0.5]$ even in high noise, high P scenarios ($P=120$, $\alpha = 0.01$, $\text{CoV} = 0.52$). This is explained by the statistically smoothing effect introduced above, in Paragraph 10.5.1: by having a larger number of fragments (above 50 in this configuration), the variations in the service times are balanced across all fragments. Even if a few experienced many retransmissions, they will be compensated by those being delivered on the first try. Therefore, the packet delay becomes more predictable relative to its mean, and its variability must be lower as well.

Following this reasoning, one may expect that the CoV of the packet delay decreases below the saturation point, following the fragments trend. Indeed, from $P = 20 \rightarrow P = 60$ in *Industrial* low noise, a CoV reduction of 3.4% in

fragment response time corresponds to a 10.4% decrease in the packet delay one. For this scenario, indeed, $P=60$ is the saturation point.

Response time *Industrial* CoV The response time CoV is decreasing below the saturation point even though the queue mechanics are much different from before. Due to high fragmentation, we find that the waiting time composes about 95% of the fragment response time: the system is queue dominated, something that was missing in the *Sensors* configuration. Therefore, increasing P below the saturation point helps reducing this congestion and the protocol overheads. From $P = 20 \rightarrow 60$ in *Industrial* low noise, the waiting time is reduced by 25%. Even though the service time increases as expected, it counts only for about 1% of the response time, so it is negligible. The response time is therefore decreased of about 23%: queue congestion is reduced while retransmissions are still not impacting. The mean decreases by 24% and the std by 27%: the CoV decreases as well because there is less variability and the std is decreasing faster than the mean, as found in *Sensors* configuration.

Therefore, the only difference is in how the response time statistics transpose to the packet ones.

Above saturation, the trend is the same described above for *Sensors*: the CoV of the packet delay grows faster than the CoV of the fragments response time because of the increased service time variance and the Head of Line blocking queue effect, thereby undermining the statistical smoothing effect.

As shown in Figure 12, the packet delay CoV remains consistently lower than the fragment response CoV, confirming the benefit of statistical smoothing in high-fragmentation scenarios. The divergence from Figure 11 is immediate.

10.5.3 CoV and system stability

We have discussed that *Sensors* with $P=80$ in high noise causes a CoV equal to 1, while in *Industrial* high noise with $P=120$, CoV is 0.52 (with a 37% increase from $P=60$): the smoothing phenomenon discussed before provides a protective buffer for the packet delay variability.

However, low variability should not hide the increasing trend in the packet delay: the 99th percentile increases by 26% from $P = 60 \rightarrow P = 120$ in low noise, and by 70% from $P = 120 \rightarrow 200$, assuming the value of 1 second, which is exactly double the inter-arrival time of packets. Consequently, it is expectable that for $P=200$ in high noise the system does not reach a steady state: it is overwhelmed by retransmissions.

Indeed, the weak point of a high fragmentation system is the much higher sensitivity to retransmission-induced congestion above saturation: when retransmissions affect one fragment, they impact a far larger number of subsequent fragments compared to *Sensors*, therefore the system is more bound to collapsing as P increases. Physically, a higher number of retransmissions means the queue depth will increase because each fragment takes more time to be served, so the response times will get higher, and the packet delay will as well.

Therefore, in low fragmentation system going beyond the saturation point within

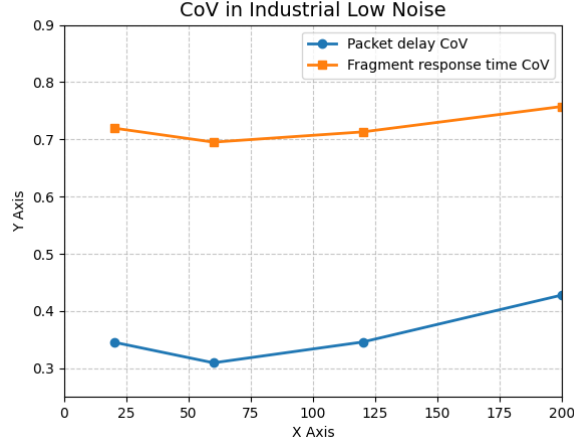


Figure 12: Comparison of CoV in the *Industrial* Low Noise scenario. The Packet Delay CoV (blue) remains significantly lower than the Fragment Response CoV (orange) due to the statistical smoothing effect of high fragmentation ($N > 50$), which averages out individual fragment variations.

reasonable limits means increasing unfairness and variability of the system. In high fragmentation systems, it means degrading system performance from a responsiveness point of view, eventually breaking possible real-time requirements and reaching instability: they are much more sensitive, showing a collapse rather than gradual degradation.

10.5.4 CoV analysis conclusions

The following conclusions can be made:

- Low-Fragmentation Systems (*Sensors*): these systems should go for P values lower than the saturation point if predictability and fairness are in the system requirements. Even though the mean delay is acceptable, the Gini coefficients, the box plots and the CoV indicate that there are too many packets experiencing disproportionately long delays, compromising real-time systems, as we can see even in the following section.
- High-Fragmentation Systems (*Industrial, Audio Streaming*): for these systems, a fragment size below the saturation point should be chosen as a buffer for the system stability, not to incur in high probability errors which cause the queue depth to spike, impacting delays.

In both cases, the choice comes at the expenses of system and protocol efficiency.

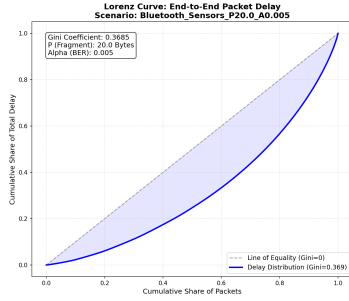
10.5.5 Lorenz Curve Analysis

As a measure of the system fairness regarding packet end-to-end delay, the Lorenz Curve analysis may be exploited, even to ensure that our CoV analysis-induced conclusions are confirmed.

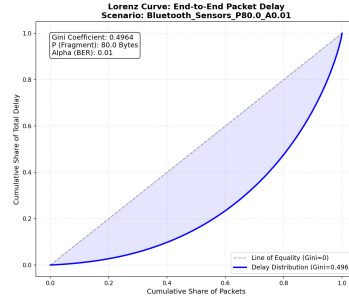
Lorenz Curve Analysis *Sensors* The Lorenz curve analysis confirms the continuous loss of fairness in packet treatment. Using the Gini coefficient as a measure for system fairness, and focusing on the *Sensors* configuration, the following was found:

- Low noise scenario: the Gini coefficient remains stable at 0.3, 0.4, meaning that most packets experience similar delays.
- High noise scenario: the Gini coefficient spikes above 0.5. The service degradation becomes non-uniform: approximately half the packets experience relatively low delays while the other half faces significantly longer delays. This could be expected with a CoV of 1.0: the Gini coefficient of an exponential distribution is indeed 0.5.

This confirms that the underlying delay phenomenon is shifting to an unpredictable, unfair service: this is seen graphically in Figure 13.



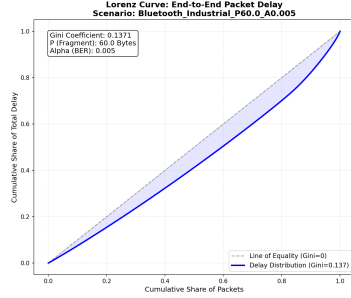
(a) Lorenz Curve of the fairest scenario in *Sensors*.



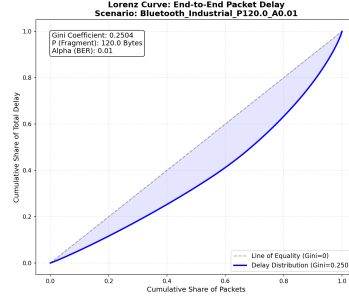
(b) Lorenz Curve of the unfairer scenario in *Sensors*.

Figure 13: These Lorenz curves graphically show how the curve deepens by increasing P and α .

Lorenz Curve Analysis *Industrial* Instead, for the *Industrial* scenario, the system becomes fairer as we approach the saturation level, as expected from the CoV analysis. Moreover, having lower packet delay CoV, the Gini coefficient never goes over 0.25: this is seen graphically in Figure 14.



(a) Lorenz Curve of the fairest scenario in *Industrial*.



(b) Lorenz Curve of the unfairer scenario in *Industrial*.

Figure 14: For the saturation point, Gini coefficient is minimized. Beyond saturation, it increases, and more if the error is high.

10.6 Bluetooth AudioStreaming

As said in this configuration description, this is the scenario which mostly resembles an ideal one: as such, the analysis was more interesting and providing more insights in the other scenarios, where many tradeoffs were to be done to satisfy system requirements. Naturally, the same trends observed before hold even in this case, thus confirming our conclusions and analysis. As seen in Figure 15, the U-shape holds, with the same considerations already made before about system equilibrium. Interestingly, the optimal point for low noise is $P=240$, a value much different from the previous configurations, due to a very low byte error rate. The CoV has the same trend already discussed: this configuration, however, poses itself in the middle of the other two. The number of fragments is between 5 and 20: therefore, we expect higher CoV values with respect to *Industrial* and lower in comparison with *Sensors*. Indeed, considering the saturation point for low noise across these configuration, we get the following values: 0.72 for *Audio Streaming*, 0.31 for *Industrial* and 0.8 for *Sensors*. Moreover, even with $P=300$ in high noise *Audio Streaming* (worst case), CoV is equal to 0.78, thus still far from 1 even if well above the saturation point ($P=120$).

At the end, this scenario benefits from partial statistical smoothing, to prevent the memory-less-like collapse seen in *Sensors*, and the queue collapse seen in *Industrial*: it represents a balanced configuration which cannot achieve high predictability but is not vulnerable to high sensitivity to beyond saturation values of P , without any utilization or delay spikes. For example, between the saturation point in *Industrial* high noise ($P=60$) and $P=120$, the 99th percentile increases by 129%, whereas between the saturation point in *Audio Streaming* high noise ($P=120$) and $P=240$, the 99th percentile increases by 17%: this trend is shown in Figure 16.

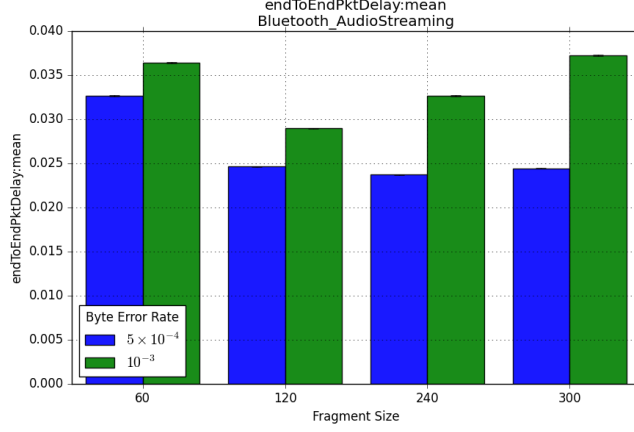
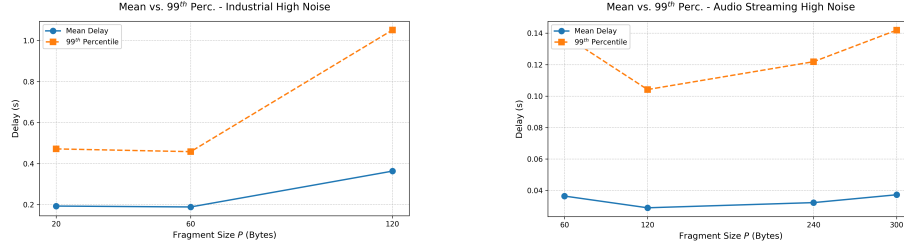


Figure 15: Packet delay of *Audio Streaming* across different scenarios



(a) *Industrial* trend of mean and 99th percentile in high noise: notice the spike after the saturation point.

(b) *Audio Streaming* trend of mean and 99th percentile in high noise: the degradation is not sudden.

Figure 16: Increased stability of *Audio Streaming* packet delay due to less fragmentation.

10.7 Queue Length estimation

For each scenario, we determined the maximum number of fragments in the queue across the 30 replicas. This allows us to estimate the necessary queue length to ensure that no fragments are lost and that we maintain the initial assumption of an infinite FIFO queue. The results have been aggregated in Table 2. In the *Industrial* scenario, the queue depth is significant and sees some important variations across scenarios. A modern Bluetooth IoT sensor, such as the reference Nordic nRF52840 SoC will handle it perfectly, with a RAM of 256 kB [19]. However, legacy or ultra-low-power sensors with more constrained resources may find the upper extremes unmanageable. This represents yet another tradeoff to make: for noisier environments, such devices may be forced to utilize smaller values of P , thereby accepting reduced protocol efficiency to

Table 2: Global Peak Queue Depth Analysis across scenarios

Scenario	P	α	$N_{q,\max}$	Queue dimension (KB)
Sensors	20	0.005	68	1.3
	20	0.010	108	2.1
	40	0.005	35	1.4
	40	0.010	52	2.0
	60	0.005	24	1.4
	60	0.010	41	2.4
	80	0.005	26	2.0
	80	0.010	50	3.9
Audio Streaming	60	0.0005	480	28.1
	60	0.0010	622	36.4
	120	0.0005	212	24.8
	120	0.0010	253	29.6
	240	0.0005	115	27.0
	240	0.0010	145	34.0
	300	0.0005	103	30.2
	300	0.0010	116	34.0
Industrial	20	0.005	2245	43.8
	20	0.010	2169	42.4
	60	0.005	640	37.5
	60	0.010	907	53.1
	120	0.005	369	43.2
	120	0.010	614	72.0
	200	0.005	249	48.6
	200	0.010	—	—

prevent buffer overflows.

In the *Sensors* scenario, the queue depth remains minimal, well within the capabilities of a general-purpose sensor. For instance, the reference Nordic nRF52840 SoC features 256 kB of RAM, allowing it to accommodate the requisite buffer space without issues [19].

In the Audio Streaming scenario, the queue depth remains stable, suggesting a high level of stability with minimal noise impact. Although the data volume may appear significant for a transmitting device, we can expect that audio streaming applications typically maintain a buffer of the audio file being processed. These applications send data to the transmitter only when the buffer reaches a sufficiently low level, in accordance with established communication protocols.

A Verification results

Table 3: Internal Consistency Check Results aggregated on 30 replicas of *Bluetooth Audio Streaming*

Check	Avg Error (%)	Max Error (%)
Little's Law (system): $E[N] = \lambda_{\text{frag}} \cdot E[R]$	0.0169	0.0353
Little's Law (queue): $E[N_q] = \lambda_{\text{frag}} \cdot E[W]$	0.0169	0.0353
Utilization law: $\rho = \lambda_{\text{frag}} \cdot E[t_s]$	0.0165	0.0340
Response time: $E[R] = E[W] + E[t_s]$	0.0000	0.0000
System occupancy: $E[N] = E[N_q] + \rho$	0.0000	0.0000
Flow balance: $\gamma = \frac{\bar{S} \cdot 8}{E[T]}$	0.0422	0.0920

Table 4: Continuity Check: Packet Size Trend

$\bar{S}$	Dev %	Thru Dev %	Avg Thru	Delay Dev %	Avg Delay (s)	Avg queue dim.	Avg ρ
350.0	+0.00%	+0.00%	2797.23	+0.00%	0.003968	0.0072	0.0041
352.5	+0.71%	+0.68%	2816.35	+0.72%	0.003996	0.0073	0.0041
355.0	+1.43%	+1.40%	2836.45	+1.43%	0.004025	0.0074	0.0042
357.5	+2.14%	+2.19%	2858.63	+2.15%	0.004053	0.0075	0.0042
360.0	+2.86%	+2.87%	2877.62	+2.87%	0.004082	0.0076	0.0042

Table 5: Continuity Check: Inter-Arrival Time Trend

$E[T]$	Dev %	Thru Dev %	Avg Thru	Delay Dev %	Avg Delay (s)	Avg queue dim.	Avg ρ
1.0000	+0.00%	+0.00%	2797.23	+0.00%	0.003968	0.0072	0.0041
1.0025	+0.25%	-0.24%	2790.38	-0.00%	0.003968	0.0072	0.0041
1.0050	+0.50%	-0.50%	2783.35	-0.00%	0.003968	0.0071	0.0041

Table 6: Continuity Check: α (Low Utilization)

α	Dev %	Thru Dev %	Avg Thru	Delay Dev %	Avg Delay (s)	Avg queue dim.	Avg ρ
0.00100	+0.00%	+0.00%	27946.64	+0.00%	0.004519	0.0843	0.0456
0.00105	+5.00%	+0.05%	27959.85	+0.61%	0.004546	0.0849	0.0459
0.00110	+10.00%	+0.04%	27958.05	+1.22%	0.004574	0.0854	0.0461

Table 7: Continuity Check: α (High Utilization)

α	Dev %	Thru Dev %	Avg Thru	Delay Dev %	Avg Delay (s)	Avg queue dim.	Avg ρ
0.00100	+0.00%	+0.00%	27954.01	+0.00%	0.203652	6.5244	0.7599
0.00105	+5.00%	+0.00%	27954.60	+2.25%	0.208233	6.6983	0.7643
0.00110	+10.00%	-0.04%	27944.22	+4.24%	0.212291	6.8484	0.7683

B Theoretical Queuing Model Computations

We model the system as an M/G/1 queue with group arrivals.

B.1 Input Parameters

C	Bandwidth (bps)
P, H, A	Fragment, Header, and ACK sizes (Bytes)
α	Byte Error Rate
$\bar{T}$	Mean inter-arrival time (s) $\implies \lambda_g = 1/\bar{T}$, arrival rate of packets
$S_{\min}, S_{\max}$	Packet size range (Bytes)

B.2 Fragment Statistics

We model the number of fragments per packet, K , as a random variable. Initially, a deterministic approximation was assumed, with the mean number of fragments per packets was computed as $E[K] = \lceil \frac{E[S]}{P} \rceil$. However, this simplified model failed to account for a large disparity between $S_{\min}$ and $S_{\max}$ because it inherently assumes that packet dimension is effectively represented by the mean packet size. At the end, severe errors were introduced in the queuing metrics. This approach, however, is accurate when packets have a fixed size, i.e. $S_{\min} = S_{\max}$, and when $P \approx S_{\max} - S_{\min}$, conditions under which the variability in the number of fragments per packet is sufficiently minimized. As a consequence, a different approach was chosen.

B.2.1 Case 1: Fixed Packet Size ($S_{\max} = S_{\min}$)

$$E[K] = \left\lceil \frac{S_{\max}}{P} \right\rceil, \quad E[K^2] = (E[K])^2 \quad (1)$$

B.2.2 Case 2: Uniform Distribution ($S_{\min} < S_{\max}$)

Defining $\Delta S = S_{\max} - S_{\min}$ and $K_{\max} = \lceil S_{\max}/P \rceil$, the Probability Mass Function (PMF) for $k \in \{1, \dots, K_{\max}\}$ is determined by the proportion of the packet size range that can be covered by exactly k fragments, relative to the total width of this range ΔS .

$$\Pr(K = k) = \frac{\min(k \cdot P, S_{\max}) - \max((k-1) \cdot P, S_{\min})}{\Delta S} \quad (2)$$

The first two moments are:

$$E[K] = \sum_{k=1}^{K_{\max}} k \cdot \Pr(K = k) \quad (3)$$

$$E[K^2] = \sum_{k=1}^{K_{\max}} k^2 \cdot \Pr(K = k) \quad (4)$$

B.3 Service Time (t_s)

The service time is defined as the time interval in which a fragment is the object of transmission, so from the moment its first transmission is started, until receiving an ACK from the receiver; after that, a new fragment may be object of transmission if the queue is not empty. It is therefore composed of a certain number of transmission cycles. The transmission cycle is defined as the time interval between the beginning of a fragment transmission and the receiving of an ACK or a NACK.

B.3.1 Transmission Cycle

With $\bar{S} = (S_{\min} + S_{\max})/2$ as the mean packet size and $\overline{S_{frag}} = \frac{\bar{S}}{E[K]}$ as the average fragment size, the average transmission cycle time per fragment is:

$$t_{\text{cycle}} = \frac{8 \cdot (\overline{S_{frag}} + H + A)}{C} \quad (5)$$

Multiplication by 8 is necessary for bytes to bits conversion, as C is measured in bits per second.

B.3.2 Error and Retransmission Statistics

The fragment corruption probability p depends on the average vulnerable size:

$$\bar{S}_{\text{vuln}} = \overline{S_{frag}} + H \quad (6)$$

$$p = 1 - (1 - \alpha)^{\bar{S}_{\text{vuln}}} \quad (7)$$

$$p_{\text{full}} = 1 - (1 - \alpha)^{P+H} \quad (8)$$

$$p_{\text{diff}} = \frac{p_{\text{full}} - p}{p} \cdot 100\% \quad (9)$$

p_{full} is the fragment corruption probability of the full size fragments, while p is the one computed with the average vulnerable size. Therefore, p_{diff} is the percentage difference between the two. It is computed as a evidence to explain the divergence of simulation measured values from the theoretical ones.

We assume that transmissions are independent and that each of them has a probability of success of $1 - p$ (i.e. the fragment is delivered uncorrupted).

Therefore, we model the transmission of a fragment as a geometric random variable G counting the number of transmissions necessary till getting an ACK.

$$E[G] = \frac{1}{1-p}, \quad \text{Var}(G) = \frac{p}{(1-p)^2}, \quad E[G^2] = \text{Var}(G) + (E[G])^2 \quad (10)$$

B.3.3 Service Time Moments

τ is the residual service time (taken from [1]).

$$E[t_s] = t_{\text{cycle}} \cdot E[G] \quad (11)$$

$$E[t_s^2] = t_{\text{cycle}}^2 \cdot E[G^2] \quad (12)$$

$$E[\tau] = \frac{E[t_s^2]}{2 \cdot E[t_s]} \quad (13)$$

B.4 System Performance Metrics

The system utilization is defined as $\rho = \lambda_g \cdot E[K] \cdot E[t_s]$, with $\lambda_g \cdot E[K]$ representing the fragments arrival rate. For stability, $\rho < 1$.

B.4.1 Mean Waiting Time ($E[W]$)

Pollaczek-Khinchin formula cannot be used in this context due to batch arrivals: this formula, extracted from [1], correctly models an M/G/1 system with batch arrivals. The first term at the right-hand side is equal to the mean waiting time in the system where customers arrive one by one according to a Poisson process with rate $\lambda_g \cdot E[K]$. The second term indicates the extra mean delay due to clustering of arrivals.

$$E[W] = \frac{\rho \cdot E[\tau]}{1 - \rho} + \frac{(E[K^2] - E[K]) \cdot E[t_s]}{2 \cdot E[K] \cdot (1 - \rho)} \quad (14)$$

From this, applying Little's Law and exploiting the additivity of mean values, the following metrics can be computed:

$$E[N_q] = (\lambda_g E[K]) \cdot E[W] \quad (\text{Avg fragments in queue}) \quad (15)$$

$$E[R] = E[W] + E[t_s] \quad (\text{Avg response time}) \quad (16)$$

$$E[N] = (\lambda_g E[K]) \cdot E[R] \quad (\text{Avg fragments in system}) \quad (17)$$

As a note, from equation (14), we see that if the deterministic approach for K is used, due to $\text{Var}(K) = 0$, $E[K^2] = E[K]^2$, so it simplifies to:

$$E[W] = \frac{\rho \cdot E[\tau]}{1 - \rho} + \frac{(E[K] - 1) \cdot E[t_s]}{2 \cdot (1 - \rho)} \quad (18)$$

C Validation results for simulated scenarios

This appendix contains the detailed numerical validation for all simulated scenarios, comparing theoretical expectations with simulation measurements. Given a simulation measurement m and its corresponding theoretical expectation e , the error ϵ is defined as:

$$\epsilon = \frac{|m - e|}{|e|} \cdot 100\% \quad (19)$$

Two example tables have been uploaded: a nearly perfect validation, and one showing very high errors, as support for the claims made in 8. The simulation results are the average of the values measured in 30 replicas of the same scenario.

Table C.1: Validation Results for Scenario: Bluetooth Industrial (P=20.0, $\alpha=0.01$)

Metric	Theory	Simulation	Error (%)
λ	701.000000	702.223495	0.17
μ	2234.827657	2234.523409	0.01
ρ	0.313671	0.314629	0.31
$E[K]$	350.500000	350.511202	0.00
$E[G]$	1.365163	1.365193	0.00
$E[N]$	80.813164	81.205535	0.49
$E[N_q]$	80.499493	80.890901	0.49
$E[R]$	0.115283	0.115498	0.19
$E[W]$	0.114835	0.115050	0.19
$p_{diff} = +0.08\%$			
$\overline{S_{frag}} = 19.97$			

Table C.2: Validation Results for Scenario: Bluetooth Sensors (P=60.0, $\alpha=0.01$)

Metric	Theory	Simulation	Error (%)
λ	750.000000	743.624753	0.85
μ	1227.368866	1142.907983	6.88
ρ	0.611063	0.650744	6.49
$E[K]$	1.500000	1.487467	0.84
$E[G]$	1.669572	1.698600	1.74
$E[N]$	1.807303	2.364342	30.82
$E[N_q]$	1.196239	1.713598	43.25
$E[R]$	0.002410	0.003179	31.92
$E[W]$	0.001595	0.002304	44.45
$p_{diff} = +27.20\%$			
$\overline{S_{frag}} = 40$			

References

- [1] Ivo Adan and Jacques Resing. Queueing systems: Lecture notes. Eindhoven University of Technology, 2015. Available at: <https://iadan.win.tue.nl/queueing.pdf>.
- [2] Apple Developer Library. *Technical Note TN2271: HTTP Live Streaming Preparation Guide*, 2016. Defines standard 128 kbps audio encoding rates.
- [3] Apple Inc. *Apple Digital Masters*, 2020. Accessed: 2026-01-03.
- [4] Apple Inc. *Logic Pro User Guide*, 2023. Section on Variable Bitrate (VBR) encoding efficiency.
- [5] Beckhoff Automation. Tf3600 — tc3 condition monitoring. https://infosys.beckhoff.com/english.php?content=../content/1033/tf3600_tc3_condition_monitoring/3394471307.html&id=, n.d. Accessed: 2026-01-03.
- [6] Bluetooth SIG. *Bluetooth Core Specification v4.0*, 2010. See p. 2200 for header dimensions, p. 1439 for 20-byte payload limits, and Attribute Op-Code/Handle definitions for overhead analysis.
- [7] Bluetooth SIG. Advanced audio distribution profile (A2DP) specification v1.3.2. Technical report, Bluetooth Special Interest Group, 2019. Adopted Specification. Available at: [bluetooth.com](https://www.bluetooth.com/specifications/a2dp).
- [8] Bluetooth SIG. *Bluetooth Core Specification v5.4: Architecture, Mixing, and Conventions*, 2023. Comparison of BLE and BR/EDR architectures.
- [9] Bluetooth SIG. *Bluetooth Core Specification v5.4: BR/EDR Controller*, 2023.
- [10] Bluetooth SIG. *Bluetooth Core Specification v5.4: Low Energy Controller*, 2023.
- [11] S. L. Cotton and W. G. Scanlon. Characterization of Body Shadowing in Indoor Radio Propagation at 2.4 GHz. In *IEEE Antennas and Propagation Society International Symposium*, pages 4143–4146, 2006.
- [12] Fraunhofer IIS. AAC: The universal audio codec for broadcast and cable TV. White paper, Fraunhofer Institute for Integrated Circuits IIS, 2019.
- [13] Fraunhofer IIS. AAC codec overview: Performance and bitrates. Technical report, Fraunhofer Institute for Integrated Circuits IIS, 2023. Scribd Archive. Describes 320 kbps high-fidelity configurations.
- [14] GraphPad Software. The confidence interval of a standard deviation, 2024. Accessed: 2026-01-14.

- [15] ISO/IEC. ISO/IEC 13818-7: Information technology — generic coding of moving pictures and associated audio information — part 7: Advanced Audio Coding (AAC), 2006.
- [16] Hesam Koushyar et al. Impact of sampling rate on the accuracy of biomechanical parameters in sports performance monitoring. *Sensors*, 25(19), 2025. Accessed: 2026-01-03. Note: High sampling rates (≥ 500 Hz) are required for accurate biomechanical parameters.
- [17] Movesense. *GATT SensorData Protocol (GSP)*. Movesense, 2023. Accessed: 2026-01-03.
- [18] Nordic Semiconductor. *Nordic Thingy:52 Firmware Architecture: Motion Service*. Nordic Semiconductor, Oslo, Norway, 2017. Accessed: 2026-01-03.
- [19] Nordic Semiconductor. nRF52840 Bluetooth 5.4 SoC. <https://www.nordicsemi.com/Products/nRF52840>, 2024. Accessed: 2026-01-03.
- [20] Nordic Semiconductor. CORE 2 integrates Nordic nRF52840 to deliver real-time performance insights to athletes. <https://www.nordicsemi.com/Nordic-news/2025/09/CORE-2-integrates-Nordic-nRF52840-to-deliver-real-time-performance-insights-to-athletes>, September 2025. Accessed: 2026-01-03.
- [21] Mark E. Payton. Confidence intervals for the coefficient of variation, 1996. Conference on Applied Statistics in Agriculture.
- [22] STATSports. How do I pair my Tracker to my phone and the STATSports Academy app? <https://support.statsports.com/hc/en-us/articles/28995152426909-How-do-I-pair-my-Tracker-to-my-phone-and-the-STATSports-Academy-app>, 2024. Accessed: 2026-01-03.
- [23] TE Connectivity. Wireless vibration sensors. <https://www.te.com/en/products/sensors/vibration-sensors/wireless-vibration-sensors.html>, n.d. Accessed: 2026-01-03.
- [24] Martin Woolley. The role of Bluetooth technology in the Ambient IoT, 2023. Accessed: 2026-01-03.